

08: Testing, CI/CD, Observability

Job descriptions for cloud software roles ask for "high-quality code through unit and integration testing" and experience with CI/CD. In an interview, that translates into questions such as "How would you test an operator without a GPU cluster?" and "What would you monitor in this service?" This chapter covers the test layers for SDKs and operators, a CI pipeline, the signals to collect in production, and load testing.

This chapter covers

- The layers of testing for cloud SDKs and Kubernetes operators, and what each layer catches
- Testing controllers with a fake client, envtest, and Kind
- Testing SDKs and CLIs, including retries and output formats
- The stages of a CI/CD pipeline
- Metrics, logs, traces, and alerts for a GPU control plane
- Load testing an API

Contents

1. Test layers	2
2. Testing a Kubernetes operator.....	3
Unit tests with the fake client.....	3
Integration tests with envtest.....	3
End-to-end tests with Kind	3
3. Testing an SDK or CLI.....	4
4. A CI/CD pipeline.....	5
5. Observability	6
Metrics.....	6
Logs	6
Traces	6
Alerts	6
Dashboards	6
6. Performance and load testing	7
Questions to expect.....	8
Summary	9

1. Test layers

Tests differ in speed, cost, and what they can detect. Use many fast tests at the bottom and fewer slow tests at the top:

End-to-end: Kind, a real cluster, a real cloud	few, slow
Integration: envtest, containers, live API	
Unit: pure logic, fake clients	many, fast

Layer	What it runs against	What it catches
Unit	Pure functions, or a controller with a fake client	Logic errors in state transitions, parsing, and object construction
Integration	A real API server and etcd (envtest), a database in a container, HTTP stubs	CRD schema errors, missing RBAC permissions, serialization problems
End-to-end	A complete cluster with the operator installed	Problems that appear only with a real kubelet, runtime, and built-in controllers
Contract	The SDK against the API's OpenAPI specification	The SDK and the server disagreeing about request or response shapes
Property-based and fuzz	Generated inputs	Edge cases in parsers and serialization that hand-written examples miss

2. Testing a Kubernetes operator

Unit tests with the fake client

controller-runtime provides a fake client that stores objects in memory. You can call the reconciler directly and inspect the objects it creates:

```
scheme := runtime.NewScheme()
clientgoscheme.AddToScheme(scheme)
gpucloudv1.AddToScheme(scheme)

c := fake.NewClientBuilder().
    WithScheme(scheme).
    WithStatusSubresource(&gpucloudv1.GpuWorkload{}).
    Build()
```

WithStatusSubresource makes the fake client treat status as a subresource, as the real API server does, so status updates must go through `Status().Update()`.

The fake client needs no cluster and runs in milliseconds. It has limits: no admission webhooks, no validation against the CRD schema, and no built-in controllers. Creating a Deployment through the fake client does not create Pods, and the Deployment never becomes available.

Integration tests with envtest

envtest starts a real kube-apiserver and etcd as local processes, without nodes or controllers.

- Install the binaries with `setup-envtest`.
- Install your CRDs, start your manager against the test API server, and assert on the resulting objects.
- envtest catches errors the fake client cannot, such as a CRD schema that rejects your object, a webhook that fails, or a field that the API server defaults.

End-to-end tests with Kind

Kind runs a Kubernetes cluster in Docker containers, with a real kubelet and containerd.

1. Create a cluster.
2. Install the CRDs and deploy the operator, or run it locally against the cluster.
3. Apply a custom resource.
4. Wait for the Deployment to become available and the Service to have endpoints.
5. Assert on the custom resource's status.

`scripts/verify-kind.sh` in this repository performs these steps. Kind has no GPUs, so use a sample resource that requests no GPUs, and test GPU-specific behavior on real hardware separately.

3. Testing an SDK or CLI

- **Stub the HTTP server.** Use wiremock in Rust, httptest in Go, or respx and responses in Python, so tests control every response.
- **Record and replay.** Capture real API responses once and replay them in tests. Remove credentials from recordings.
- **Inject failures.** Return 429, 503, and dropped connections to verify retries, backoff, and the retry limit.
- **Test token refresh.** Return 401 for an expired token and check that the client refreshes and retries once.
- **Test every output format.** Compare JSON, YAML, and table output with golden files checked into the repository.
- **Test on every supported platform.** Run the suite on Linux, macOS, and Windows with a CI build matrix, because paths, line endings, and terminal behavior differ.

4. A CI/CD pipeline

A pipeline for this repository's kind of project runs these stages, stopping at the first failure:

1. **Lint and format.** `cargo fmt --check`, `cargo clippy`, `gofmt`, `golangci-lint`.
2. **Unit tests.** `cargo test` for Rust and `go test ./...` for Go.
3. **Generated code check.** Run the code generators and fail if `git diff` shows changes, which means someone changed an API type without regenerating.
4. **Build.** Build binaries for each target platform.
5. **End-to-end tests.** Create a Kind cluster and run the operator tests.
6. **Publish.** Push container images and binaries.
7. **Release.** Tag a semantic version and publish a changelog.

The Kind job in this repository's `.github/workflows/ci.yaml` follows this shape:

```
jobs:
  kind-e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "stable"
      - uses: helm/kind-action@v1.12.0
      - run: kubectl apply -f config/crd/bases/
      - run: go test ./...
      - run: go run ./cmd/main.go &           # run the operator in the background
      - run: kubectl apply -f config/samples/gpucloud_v1_gpuworkload_kind.yaml
      - run: kubectl rollout status deployment/triton-kind-gpu --timeout=180s
```

5. Observability

Collect four kinds of signal: metrics, logs, traces, and the alerts built on them.

Metrics

controller-runtime exposes Prometheus metrics at `/metrics` by default. The most useful built-in metrics for an operator are:

Metric	What it shows
<code>workqueue_depth</code>	Objects waiting to be reconciled; a growing value means the controller is falling behind
<code>controller_runtime_reconcile_errors_total</code>	Failed reconciliations
<code>controller_runtime_reconcile_time_seconds</code>	How long each reconciliation takes

Add business metrics for the service itself, such as the number of active GPU workloads, GPUs allocated compared with allocatable, and create latency from request to Ready.

Logs

Write structured logs, such as JSON, with consistent fields: the resource name and namespace, the reconcile ID, and the tenant. Use `log/slog` or `logr` in Go and tracing in Rust. Structured fields let you filter the logs for one workload across all components.

Traces

Use OpenTelemetry to propagate a trace from the API gateway through the control plane services to the operator. A trace shows which step of a slow create request took the time.

Alerts

Alert on symptoms that need a person to act:

Alert	Condition
<code>GpuWorkloadNotReady</code>	A workload has not reached Ready within 10 minutes
<code>ReconcileErrorRateHigh</code>	More than 1% of reconciliations fail over 15 minutes
<code>GpuNodeNotReady</code>	A node running GPU workloads is NotReady
<code>GpuHealthDegraded</code>	<code>dcm-exporter</code> reports Xid or uncorrectable ECC errors, or stops reporting

Dashboards

Build Grafana dashboards from `dcm-exporter` metrics: GPU utilization, memory use, temperature, power, and ECC error counts for each node.

6. Performance and load testing

- **Generate load** with k6, locust, or hey.
- **Measure** p50, p95, and p99 latency, throughput, and error rate. Averages hide the slow requests that users notice.
- **Test the SDK under failure.** Return 429 and 5xx responses at a known rate and confirm that retries do not multiply the load.
- **Profile** Go with pprof, and Rust with perf and cargo flamegraph.
- **Common improvements** include connection pooling, faster JSON encoding and decoding, smaller page sizes, and caching of read-heavy endpoints.

Questions to expect

- How would you test an operator without a GPU cluster?
- A test for the SDK fails intermittently. How do you make it reliable?
- What metrics would you collect for a GPU cloud control plane?
- How would you roll out an incompatible API change?
- Which behaviors belong in unit tests, and which need end-to-end tests?

Summary

- Use unit tests for logic, integration tests for API server behavior, and a small number of end-to-end tests for the full system.
- The controller-runtime fake client is fast but does not run webhooks, schema validation, or built-in controllers. envtest and Kind fill those gaps.
- Test SDKs against stubbed servers that inject failures, and test CLI output with golden files.
- A CI pipeline should fail when generated code is out of date.
- Monitor work queue depth, reconcile errors, and reconcile duration for operators, and alert on symptoms that require action.

Source: [08-testing-cicd-observability.md](#) · [PDF](#) · [All study guides](#)